

Interface Intelligence Definition Pack

A recommended product definition for an agentic transaction lineage layer across every client integration, API call, and system-to-system handoff in the revenue and customer lifecycle.

681 QTR SCENARIO RECORDS	47 CONTROLS	20 AI VALIDATION TESTS	2 LINEAGE TOKENS	1 INTERFACE LEDGER
--------------------------------	----------------	------------------------------	---------------------	-----------------------

Recommended name **HandoverOS Interface Intelligence Layer** - the observable transaction ledger and agent analysis system that turns invisible cross-system movement into explainable operational truth.

Definition Every interface transaction becomes a source-traced event with field values, transformations, waterfall steps, validation results, lifecycle tokens, customer-journey tokens, evidence status, and agent-readable lineage.

Prepared July 2026 from Gmail connector analysis, HandoverOS/QTR workbook evidence, Salt Basin agent memory files, current website lineage code, and attached Salt Basin visual design standards.

1. Executive Definition

The product should be defined as an agentic interface intelligence layer, not a generic integration monitor. Its job is to capture the factual movement of data between systems, preserve the transformation logic, and make the business meaning of every handoff explainable.

Product thesis

Most revenue leakage and customer-friction issues are not created at the point where they become visible. They are created upstream in a field, rule, approval, transform, missing token, stale source, or handoff that no one can see later.

Recommended one-line definition

HandoverOS Interface Intelligence traces every revenue and customer lifecycle data transaction across systems, then uses agents to explain lineage, detect mismatches, identify hidden operational risk, and create reusable institutional memory.

What this product is

- A transaction ledger for every interface, API call, integration job, webhook, batch file, connector, and system-to-system handoff.
- A field-level lineage model that stores source values, transformed values, validation steps, business rules, and downstream object links.
- An agent analysis layer that can answer why a value changed, where a mismatch started, what controls failed, and which lifecycle outcome is at risk.
- A governance layer that separates verified facts, modeled assumptions, stale benchmarks, proprietary context, and public-safe summaries.

What this product is not

- Not only logging. Logging says what happened technically; interface intelligence explains operational meaning.
- Not only data lineage. Classic lineage traces tables and fields; this traces revenue promises, customer state, controls, exceptions, and evidence.
- Not only an AI copilot. The agent is useful because the ledger gives it durable, governed context.

2. Token Model

The design should use two persistent tokens that travel across systems and attach to every transaction where possible. These tokens are the product's organizing spine.

Token	Purpose	Lifecycle
Revenue Lifecycle Token	Traces the commercial object as it becomes a financial obligation and revenue stream.	prospect -> pipeline -> proposal -> contract -> order -> subscription -> bill -> invoice -> collect -> recognize -> renew/expand -> adjust
Customer Journey Token	Traces the adjacent customer/account/person experience through acquisition, onboarding, payment, service, and change.	external lead -> qualified prospect -> negotiation -> signature -> onboarding -> active customer -> payment -> support/change -> renewal/expansion/churn

Why both tokens matter

The revenue token answers 'what happened to the money, obligation, and measurement.' The customer token answers 'what happened to the relationship, experience, and promise.' Many failures only become clear when both are viewed together.

Primary questions the tokens should answer

- Where did this ARR, invoice amount, term date, discount, tax treatment, entitlement, or renewal value originate?
- Which interface or transformation changed it?
- Did the contract, order, billing, recognition, and customer-success records preserve the same business promise?
- Which customer-facing event was adjacent to the financial error: onboarding, usage, dispute, service request, change request, renewal, or cancellation?
- Is the mismatch an isolated defect, a recurring handoff pattern, or a governance/control weakness?

3. Transaction Ledger Schema

The ledger should be event-first. Every connector reports a normalized transaction event, even when the underlying systems use different object names, payload shapes, or lifecycle labels.

Schema area	Recommended fields
Identity	transaction_id, correlation_id, source_event_id, idempotency_key, tenant_id, profile_scope, profile_id
Systems	interface_id, connector_id, source_system, target_system, environment, api_endpoint, job_name
Objects	source_object, source_record_id, target_object, target_record_id, parent_record_id, external_reference_ids
Tokens	revenue_lifecycle_token, customer_journey_token, account_token, contract_token, subscription_token
Payloads	payload_before_hash, payload_after_hash, redacted_payload_sample, changed_field_count
Field lineage	field_path, source_value, transformed_value, target_value, transform_rule, confidence, required_flag
Waterfall	step_number, step_name, actor_type, validation_result, control_result, error_code, retry_count
Business context	lifecycle_stage, customer_stage, revenue_model, contract_type, billing_model, process_area
Evidence and governance	source_refs, sensitivity, approval_status, benchmark_status, assumption_flag, retention_policy
Agent analysis	risk_score, anomaly_type, root_cause_hypothesis, recommended_action, human_review_required

MVP rule

Start with hashes, diffs, mappings, and IDs. Do not require storing every raw payload forever. Preserve enough evidence to replay the lineage and enough redaction to make the layer safe.

4. Agent Analysis Loop

The agent layer should be a controlled reasoning loop over the ledger. It should use Salt Basin's universal reasoning pattern: observe, question assumptions, map relationships, validate evidence, quantify uncertainty, create visibility, design governance, and produce reusable systems.

Loop step	What the agent does	Output
Observe	Reads transaction, field diffs, step sequence, linked records, and source trace.	Transaction brief
Classify	Maps event to lifecycle stage, customer stage, failure family, control type, and risk area.	Classification tags
Trace	Walks upstream and downstream through tokens and correlation IDs.	Lineage path
Explain	Separates facts, assumptions, interpretations, and likely root causes.	Explainability summary
Detect	Compares against QTR scenarios, controls, KPIs, and recurring leakage patterns.	Risk/anomaly finding
Recommend	Suggests validation, remediation, owner, control, or next evidence request.	Action packet
Remember	Stores approved reusable pattern without exposing raw sensitive data.	Memory candidate

Core specialist agents

- Interface Transaction Analyst: summarizes one event and its field-level changes.
- Revenue Lifecycle Tracer: follows the token from pipeline through proposal, contract, order, billing, recognition, renewal, and adjustment.
- Customer Journey Tracer: follows lead, onboarding, support, change, payment, and renewal context.
- Control Evidence Agent: checks whether required controls, validations, and approvals occurred.
- Root Cause Pattern Agent: maps failures to QTR scenario families and recurring handoff patterns.
- Benchmark and Claims Guard: blocks stale or unsupported external figures from public/client-facing use.
- Memory Steward: classifies new insight as public-safe, internal, restricted, confidential, or quarantine.

5. Parallel Agent Sizing

The product should not send every transaction to one giant agent. It should profile the workload, shard it by stable boundaries, run duplicate workers in parallel, and roll findings up through leads and managers.

Workload	Example volume	Recommended pool
Small	Up to 1,000 transactions, 150 fields, or one process tower	1-2 workers, optional lead, 1 orchestrator
Medium	1,000-25,000 transactions, 3-5 process towers, or 10 integrations	3-6 workers, 1-2 leads, 1 manager
Large	25,000-250,000 transactions, 5-10 towers, or many systems	8-20 workers, 3-5 leads, 2-4 managers
Portfolio	250,000+ transactions, multiple companies, or fund-level review	20-60 workers, 5-12 leads, 4-8 managers

Agent type	Recommended max per worker run	Best shard unit
Interface Transaction Analyst	250 transactions or 50 error clusters	Interface, endpoint, batch, object pair
Integration Data Mapper	150 fields, 20 objects, or 10 sample payloads	System pair or object family
Revenue Lifecycle Tracer	100 lifecycle tokens or 500 related events	Lifecycle token, customer segment, product line
Customer Journey Tracer	100 journey tokens or 40 account histories	Account, customer stage, journey phase
Control Evidence Agent	50 controls or 250 control executions	Control family, lifecycle stage
Root Cause Pattern Agent	300 defects/events or 25 anomaly clusters	Failure family, release, process area
Memory Steward	50 sensitive candidates or 250 low-risk candidates	Sensitivity class, product domain

Sizing formula

Formula

`worker_count = ceiling(total_units / max_units_per_worker).`
`functional_lead_count = ceiling(worker_count / 5).` `management_agent_count = ceiling(functional_lead_count / 3).` Add one Evidence Controller for external, legal, financial, investor, or audit-sensitive work.

6. MVP Build Path

Phase	Build	Why it matters
MVP 1	interface_transactions, interface_steps, interface_field_lineage, lineage_tokens tables plus ingestion API	Creates durable operational truth before advanced AI.
MVP 2	Admin Lineage view for searching by customer, deal, quote, contract, subscription, invoice, or token	Makes debugging valuable immediately.
MVP 3	Agent summarize/explain endpoints for transaction, token, and lifecycle views	Turns ledger data into usable answers.
MVP 4	Connector adapters for Salesforce/CPQ/CLM/billing/ERP style events	Moves from demo to client-facing diagnostic.
MVP 5	Memory approval queue and public-safe preview generator	Protects proprietary methods and creates reusable product intelligence.

Recommended current-repo fit

- Extend the existing field_lineage and data_snapshots model rather than replacing it.
- Reuse the admin Lineage panel pattern but add transaction, token, and journey views.
- Map the product to the existing Lead to Revenue Capability Model and HandoverOS/QTR memory files.
- Keep raw Gmail-derived and workbook-derived knowledge behind approval gates.

7. Source Context and Confidence

Source	What it contributed	Confidence
Gmail connector live search/read	Confirmed HandoverOS/QTR attachments, outreach context, career portfolio source cluster, and QTR workbook structure.	High for source existence; medium for public claims.
QTR V5 workbook attachment	681 scenarios, 47 controls, 18 KPIs, 20 revenue leakage patterns, 20 AI validation tests, relationship mapping.	High for internal product shape.
HandoverOS attachment extraction docs	Contract metadata foundation, QTR lifecycle steps, billing/recognition separation, governance rules.	High for internal definition.
Salt Basin memory register	Guardrails for email-derived memory, benchmarks, assumptions, overclaiming, public previews.	High for agent behavior.
Current repo lineage code	Existing field-level lineage capture, snapshots, hashes, and admin API foundation.	High for implementation fit.
Visual design system PDFs/PNG	Premium executive design tokens: navy/gold palette, KPI tiles, cards, metadata chips, icon meanings.	High for output styling.

Guardrail

Benchmark values, market statistics, regulatory interpretations, monetary ROI claims, and external-source claims should be refreshed from current authoritative sources before public, client-facing, investor-facing, or decision-support use.